University of Sussex

Network Load Optimization Based on the NEAT Algorithm

Candidate No:311288

MSc Artificial Intellegence and Adaptive Systems

Supervised by Dr. Chris Johnson

Informatics Department

University of Sussex

August 2026

Declaration

This report is submitted as part requirement for the degree of MSc Artifitial Intellegence and Adaptive Systems at the University of Sussex. It is the product of my own labour except where indicated in the text. The report may be freely copied and distributed provided the source is acknowledged.

Signature:



Temir Zhumagali

Abstract

This paper is dedicated to developing a load balancing method based on NeuroEvolution of Augmenting Topologies (NEAT) algorithm which evolves the structure, weights, and biases of a neural network simultaneously. The aim of the study is to assess the effectiveness of the evolutionary topologies in the context of optimizing handover parameters for load balancing in heterogeneous and how it compares to the solutions with fixed topologies.

The work was carried out in the context of heterogeneous LTE/5G networks and is limited to simulation modeling: a simplified radio environment model is used (free space, standard thermal noise, absence of obstacles), random assignment of user velocity, direction of movement, traffic consumption, positions of the stations does not reflect thir placements in the real world.

For methodology, a network with LTE and 5G stations was deployed. The cell individual offset value, which is responsible for handovers, of each station are controlled by the neural network evolved using NEAT. Different configurations of evolutionary parameters were tested and the resulting models were compared with RL Attiah et al. (2020) and automatic weight function Gures et al. (2023) approaches.

The results showed that the fitness function values depend weakly on the tuning of NEAT hyperparameters and on the use of recurrent connections. Compared to RL and AWF, the NEAT algorithm demonstrated a higher degree of user and traffic satisfaction with significantly less execution time. At the same time, an analysis of the structure of the evolved networks revealed that the models rely almost exclusively on the station type, ignoring other input features, which indicates that the found solution is a simplified heuristic with limited generalization capacity. Overall, the results suggest that the NEAT approach can be a viable alternative to methods with fixed topology and AWF approaches for load balancing in heterogeneous networks, however this statement requires further verifications because the effectiveness of the system in other simulation conditions is unknown.

Acknowledgements

I express my gratitude to my supervisor Chris Johnson for the guidance, valuable advice and constructive feedback throughout this research. Dedicated support was important in the development and completion of this dissertation. I would also like to acknowledge the High Performance Computing facilities for providing the computational resources that facilitated computations.

Contents

List of Tables vii

List of Figures viii

1 Introduction 1

2 Literature review 3

3 Methodology 5

3.1 Metrics . . . 5

3.2 NEAT . . . 7

3.2.1 Genotype and phenotype . . . 8

3.2.2 Mutation . . . 8

3.2.3 Crossover . . . 9

3.2.4 Speciation and protection of structural innovations . . . 10

3.2.5 Minimal initialization of the structure . . . 11

3.3 Changes to NEAT . . . 11

3.3.1 Accumulation of Weak, Near-Zero Connections . . . 11

3.3.2 Loss of Elite Genomes . . . 12

3.3.3 Weight Incompatibility Between Matching Genes . . . 12

3.3.4 Inheritance Bias Toward the Fitter Parent . . . 13

3.4 Proposed optimization method . . . 13

3.4.1 Model architecture . . . 13

3.4.2 Resource Allocation Model . . . 16

3.4.3 Serving-Cell Selection and Handover Algorithm . . . 17

3.4.4 Neural network controlled CIO . . . 18

3.5 Other algorithms . . . 19

3.5.1 Automated Weight Function algorithm . . . 19

3.5.2 RL-based model 22

4 Results **26**

4.1 Performance of NEAT-based load balancing algorithm 26

4.1.1 Simulation setup 26

4.1.2 NEAT N1 27

4.1.3 NEAT N2 28

4.1.4 Recurrent neural network (RNN) 30

4.1.5 Random simulation testing 30

4.1.6 Fitness and speciation during static testing 31

4.1.7 Analysis of the performance of the NEAT-based load balancing algorithm 32

4.2 Comparison with reinforcement learning and automatic weight function methods 33

4.2.1 Analysis of the performance of proposed algorithm compared to Rl-based and AWF-based methods 34

4.2.2 Limitations 34

5 Conclusion **35**

Bibliography **37**

A Appendix **40**

A.1 Instruction for running the code 40

A.2 Topology examples 40

List of Tables

3.1 LTE and 5G Network Simulation Parameters 14

4.1 Simulation parameters 26
4.2 NEAT configuration N1 parameters 27
4.3 NEAT configuration N1 results. Traffic is not included in the fitness function because it is a reflection of satisfaction. 28
4.4 Selected NEAT mutation parameters 29
4.5 NEAT configuration N2 results 29
4.6 Recurrent neural networks 30
4.7 Results of random testing of non-recurrent models 30
4.8 Comparing proposed algorithm with RL-based and AWF-based in random simulation conditions 33
4.9 Comparing proposed algorithm with RL-based and AWF-based in static simulation conditions 33

List of Figures

3.1 Figure was reproduced according to Stanley and Miikkulainen (2002). Matching up genomes for different network topologies using innovation numbers. Red squares represent disabled connections 10

3.2 The construction of the map was inspired by Gures et al. (2023). Green triangles represent underloaded LTE stations, red triangles represent LTE station with load over 0.7, squares represent 5G stations, green transparent circles - radius of LTE stations, yellow transparent circles - radius of 5G stations, purple circles - connected users, red circles - disconnected users and black lines - connection between the user and the station. 15

3.3 Graphical representation of the network. Gray squares represent input neurons, empty circles are hidden neurons and blue circle is output. Green lines have positive weights and red lines - negative. Dotted lines are disabled connections and straight lines are active. As presented in the figure, only one connection to the output is active and it is coming from the node that itself is receiving signals only from input -1. 25

4.1 Behavior of the users during static simulation 31

4.2 Behavior of the users during random simulation 31

4.3 Average fitness over generations 32

4.4 Average speciation over generations 32

Chapter 1

Introduction

The average data volume required by clients is steadily increasing mainly due to the higher demand for video content Attiah et al. (2020). Video content is becoming more prevalent in all types of online platforms, thus this direction is predicted to continue.

The total load on 4G networks is not uniformly distributed among cells. As stated by Holma H. (2012), it is expected for 15% of cells to carry 50% of the total traffic load. This trend creates locations with high resource utilization, known as hot spots, simultaneously with locations with lower traffic usage compared to the rest of the network. This introduces the need for more efficient offloading of overutilised cells and developing load balancing methods to manage traffic among neighboring cells that produces minimal impact on radio channels.

The heterogeneous nature of 5G networks that utilizes cells with vastly diverse ranges, power and bandwidth introduces additional problems. They must be densely deployed and highly dynamic to ensure high quality communication service required by modern users as they put an immense traffic load on the network Ansari et al. (2017). One of the most prevalent problems is effective load distribution between cells. Load balancing methods address this problem by equally distributing cell loads across neighboring cells, ensuring effective handling of higher data rates and increased system capacity. Small cells are implemented to increase capacity in hot spots and reduce the load on congested cells. Traditional systems of allocating users based on received power lead to load imbalance due to the difference in power between macro and small cells. Users tend to be connected to macro cells with higher power and share insufficient traffic while small cells remain underutilised, thus wasting resources.

Modern networks support self-optimization methods, increasing possibilities for load balancing. It had to increase in numbers of proposed laid balancing algorithms based on

handover (HO) parameters. Lobinger et al. (2010) propose an algorithm to modify HO parameters, known as cell individual offset (CIO) of overloaded serving cells in relation with neighboring cells. CIO is a bias that affects HO decisions.

A number of CIO optimising algorithms involving reinforcement learning Attiah et al. (2020), supervised learning Franco and De Marca (2015) and other machine learning techniques has been introduced. However, most existing methods rely on fixed neural network topologies or predefined policy structures. This limits their capacity to adapt to a highly dynamic environment of heterogeneous networks. Therefore, this paper introduces dynamic load balancing algorithm based on NeuroEvolution of Augmenting Topologies (NEAT). NEAT is an evolutionary algorithm that evolves topology of the neural network along with weights and biases Stanley and Miikkulainen (2002). It is designed to discover the most efficient network structure for specific tasks. Authors declare that NEAT outperforms fixed topology networks on the pole balancing task due to several advantages, such as employing a principled method of crossover of different topologies,protecting structural innovation using speciation, and incrementally growing from minimal structure.

This study focuses on exploring previously unstudied topic of utilizing NEAT for dynamic handover algorithm. The goal is to investigate whether evolutionary topology can provide more stable and efficient network behavior compared to conventional optimization approaches.

This study addresses the following research questions:

- Can a NEAT-based controller effectively optimize handover parameters for load balancing in heterogeneous cellular networks?
- Does NEAT improve load balancing performance compared to conventional fixed-topology neural networks?

This research has several limitations. The model is created and evaluated in a simulated environment with simplified noise conditions, assuming standard temperature of 290K and no other noise sources, such as obstacles. The modeled environment does not reflect real world locations of base stations. Users move randomly, without dedicated paths, such as roads or walkways.

Chapter 2

Literature review

In the paper written by Attiah et al. (2020), authors present a reinforcement learning approach to load optimization problem. The proposed method optimizes cell relational parameters to balance traffic across multiple cells. Authors present a learning system, including reward system and main performance indicators. According to the results of the research, using a reinforcement learning approach for load optimization can drastically improve system performance.

The method proposed in the paper does not directly connect and disconnect users from the system but rather adjusts CIO parameters of each tower. CIO value artificially adjusts RSRP of the cells. After that the user compares the adjusted RSRP of the neighboring cells and connects itself to the best cell.

The system provided by researchers was tested on a small network of 3 cells. Because the output layer for the RL is defined as L^N where N is the number of stations and L is the number of possible CIO values. This approach is inefficient because the output layer of the network expands exponentially with increasing number of base stations. For example, the system with 10 stations and 5 CIO values will have an action space of 9 765 625 possible actions.

The method introduced by Attiah et al. (2020) will be used for comparison with proposed algorithm. For comparison, all the cell will be divided into groups of 1 LTE station and 3 corresponding 5G stations.

Other algorithm to compare with is proposed by Gures et al. (2023) and based on Automatic Weight Function (AWF) and adjustable Time-To-Trigger (TTT). This method dynamically computes handover control parameters for each user based on SINR, PRB and user velocity. Those values are converted to auto-weights controlling artificial adjustments to RSRP of the neighboring towers and TTT which controls how long should user stay

connected before switching to the new station.

In essence, the method personalizes TTT and perceived RSRP of the stations for each user instead of applying static thresholds across the network.

Neuroevolution methods commonly use a fixed size and topology networks, in which both are predefined before training. In this type of network evolution optimizes only the connection weights via mutation and crossover. In contrast, Stanley and Miikkulainen (2002) state that topology of NN also influences its efficiency. Evolving topology increases efficiency and lowers the complexity of the network. According to Gruau et al. (1996), evolutionary topology can save time wasted on choosing appropriate structure for a particular task. As evidence, Gruau et al. evolved topology that solved the hardest pole-balancing problem for that time. However Gomez and Miikkulainen (1997) proposed a fixed topology method that could solve the same problem 5 times faster. Stanley and Miikkulainen (2002) came to the opposite conclusion, providing the NeuroEvolution of Augmenting Topologies (NEAT) method, that uses structure of the network for minimising search space of the weights. Stanley and Miikkulainen (2002) improved the efficiency of evolutionary topology by incremental growth, thus minimizing topologies over the entire evolution process, rather than at the end of it.

The paper written by Lyubchev and Marchev (2025) introduces several changes to the algorithms to improve its performance. For example, the original NEAT introduced bias in the process of crossover by inheriting all excess and disjoint genes from more fit parent. The researchers proposed the following solution: implementing controlled randomness, which means assigning probability of inheriting excess or disjoint genes from less fit parent. This method preserves selection pressure while increasing diversity. The improvements proposed by Lyubchev and Marchev (2025) are implemented for this project.

Chapter 3

Methodology

3.1 Metrics

For the project measurements such as RSRP are crucial to accurately simulate the environment. Reference Signal Received Power (RSRP) is a measurement of the average received power of resource elements carrying reference signals and is widely used in LTE and 5G networks for handover decisions European Telecommunications Standards Institute (2025). For calculating RSRP Received Signal Strength Indicator (RSSI) is used. RSSI measures the strength of the signal that user receives from a wireless source, such as cell tower. RSSI is a measurement of the signal power that has reached the user European Telecommunications Standards Institute (2025). RSSI calculated as shown in the Formula (3.2). The calculations for RSRP are described in Formula (3.1)

$$\mathrm{RSRP} = \mathrm{RSSI} - 10\log(12 * \mathrm{N}) \tag{3.1}$$

$$\mathrm{RSSI} = \mathrm{P_t} - \mathrm{L_p(d)} \tag{3.2}$$

Where

- $\mathrm{P_t}$ is the transmitter's power in dBm
- d is the distance between the user and the station
- $\mathrm{L_p(d)}$ is the path loss

The strength of the signal weakens as the distance between transmitter and receiver of the signal increases. Hence, simulation of pass loss is crucial. The calculation of the path loss is shown in Formula (3.3)

$$\mathrm{L_p(d) = L_0 + 10n \cdot \log_{10}(d/d_0)} \tag{3.3}$$

where

- L_0 is the path loss at a reference distance d_0. 1 is often used as d_0 for calculations
- n is the path loss exponent, which varies based on the environment (amount of obstacles, reflections, propagation environment, etc.). For this project $n = 2$ for free-space propagation.
- d is the distance between the transmitter and receiver

L_0 was calculated using standard Free Space Path Loss Formula (3.4)nAgarwal et al. (2026).

$$\mathrm{FSPL/L_0} = 20 \cdot log_{10}(d) + 20 \cdot log_{10}(f) - 147.55 \tag{3.4}$$

In modern networks one of the most important parameters is Signal-to-Interference-plus-Noise Ratio. SINR is central to decisions for modulation, scheduling, throughput capacity, and ultimately user experience.

SINR is defined as the ratio of the power of the desired useful signal to the sum of the power of interfering signals plus the noise power European Telecommunications Standards Institute (2025). The Formula (3.5) was user for calculating SINR.

$$\mathrm{SINR(dB)} = 10 \cdot log_{10} \cdot (\frac{S}{I + NOISE}) \tag{3.5}$$

where

- S = Received useful signal power
- I = Interference power (from other cells, UEs, sources)
- $NOISE$ = Noise power (thermal noise)

The random motion of electrons in any electronics produces the noise. According to Farré et al. (2025), the noise is present in any resistor, cable, and component, and cannot be eliminated except by cooling . The standard temperature noise formula used in this project is shown in Formula (3.6):

$$\mathrm{NOISE = k \cdot T \cdot B} \tag{3.6}$$

where:

- $k = 1.38 \times 10^{-23}\,\mathrm{J/K}$ (Boltzmann's constant)
- T = temperature in Kelvin
- B = bandwidth in Hz

Considering the standard reference temperature $T = 290\,\mathrm{K}$, converting Watts into dBm, and adding a standard noise figure of 6 dB, the noise is calculated as shown in Formula (3.7)

$$NOISE_{\mathrm{dBm}} = -174 + 10\log_{10}(B_{\mathrm{Hz}}) + 6 \tag{3.7}$$

3.2 NEAT

In general, evolutionary algorithms can be described in a single scheme: initialization of a population, assessment of the fitness of its members, recombination of genes, known as crossover and mutation operators, after which the cycle repeats for a given number of generations or until the stop criterion is reached.

When applying evolutionary methods to the task of building neural network architectures, a number of specific problems arise that the NEAT algorithm was designed to solve Stanley and Miikkulainen (2002):

- Is there a genetic representation that allows meaningful crossover between individuals with different neural network topologies? The problem is that when using binary or graph encoding architecture, the crossover operation between two arbitrary topologies does not have an obvious correct implementation.
- How can topological innovations, which take several generations to fully optimize, be protected from premature extinction from the population? The complication of the network structure is usually accompanied by the addition of new not optimized weights, which temporarily reduces the fitness of the individual. Without a special protection mechanism, such structural innovations will be eliminated by selection before they reach their potential.
- How can we minimize the complexity of topologies in the process of evolution without using an artificially constructed component of a fitness function that clearly penalizes complexity? Developing such a penalty function comes with the risk of overly limiting the search space.

The following key mechanisms are proposed in the NEAT algorithm to solve these problems:

- Genetic coding, which distinguishes between the genotype and the phenotype, that is, the resulting neural network used directly to assess fitness.
- Historical markers method, which provides the possibility of correct alignment of genes during crossover of individuals with different topologies.
- A speciation mechanism that distributes individuals by species based on structural similarity, which allows topological innovations to compete primarily within their own species, rather than with the entire population.
- Explicit fitness sharing, in which the value of an individual's fitness function is normalized depending on the number of its species.
- Minimal initialization of the structure, in which evolution begins with topologically simplest networks that gradually become more complex as beneficial mutations accumulate, unlike approaches with random initialization of arbitrary complexity.

3.2.1 Genotype and phenotype

The genotype of an individual in the NEAT algorithm consists of two sets of genes:

- A set of node genes that defines the input, hidden, and output nodes of the network and serves as a reference structure for link genes.
- A set of connection genes, each of which is determined by the following parameters: source node, receiver node, link weight, boolean sign of whether the link is enabled or disabled and innovation number.
- The innovation number is a key element of the scheme, providing the possibility of crossover between genotypes of different structures.

3.2.2 Mutation

In the context of neuroevolutionary algorithms, the mutation operator performs a function for structural and parametric changes in individuals. The NEAT algorithm provides for two types of structural mutations:

- Adding (or removing) a link: a new link is established between two previously unrelated network nodes. The corresponding link gene is added to the end of the genotype and receives a new innovation number.

- Adding (or removing) a node: an existing link between two nodes is marked as disabled, after which two new link genes are added to the genotype from the original source node to the new node and from the new node to the original receiver node.

For each structural mutation, a global innovation counter is incremented and assigned to the new gene; innovation numbers thus record the gene's chronology, and when the individuals are crossed, the offspring inherit the same numbers unchanged. If the same mutation occurs independently in several individuals of the same generation, it is assigned the same number by the innovation list of the current generation, thus preventing the numbers from growing exponentially. Such historical labels give the system a new opportunity: it knows exactly which genes are shared between genomes, as shown in Figure 3.2.

A global table of innovation numbers is used to maintain consistency in the numbering of innovations between members of the population. This somewhat limits the degree of asynchrony of the evolutionary process compared, for example, with population-based training methods. However, since only consistent read and write access to a single table is required, the algorithm as a whole allows parallelization.

3.2.3 Crossover

The implementation of crossover between neural network topologies is complicated by the so-called competing conventions problem: the same function implemented by a network can be encoded structurally in different ways. With the direct crossover of genotypes encoding functionally equivalent but structurally different networks, the resulting individual may lose some of the information necessary for the correct functioning of the network.

To solve this problem, NEAT uses gene alignment based on innovation numbers, similar to the alignment of homologous genes in biological crossover:

- The genotypes of two parental individuals are aligned according to the matching innovation numbers.

- Matching genes are inherited by a descendant randomly from one of the parents.

- Excess and disjoint genes are inherited from the more adaptable of the two parental individuals.

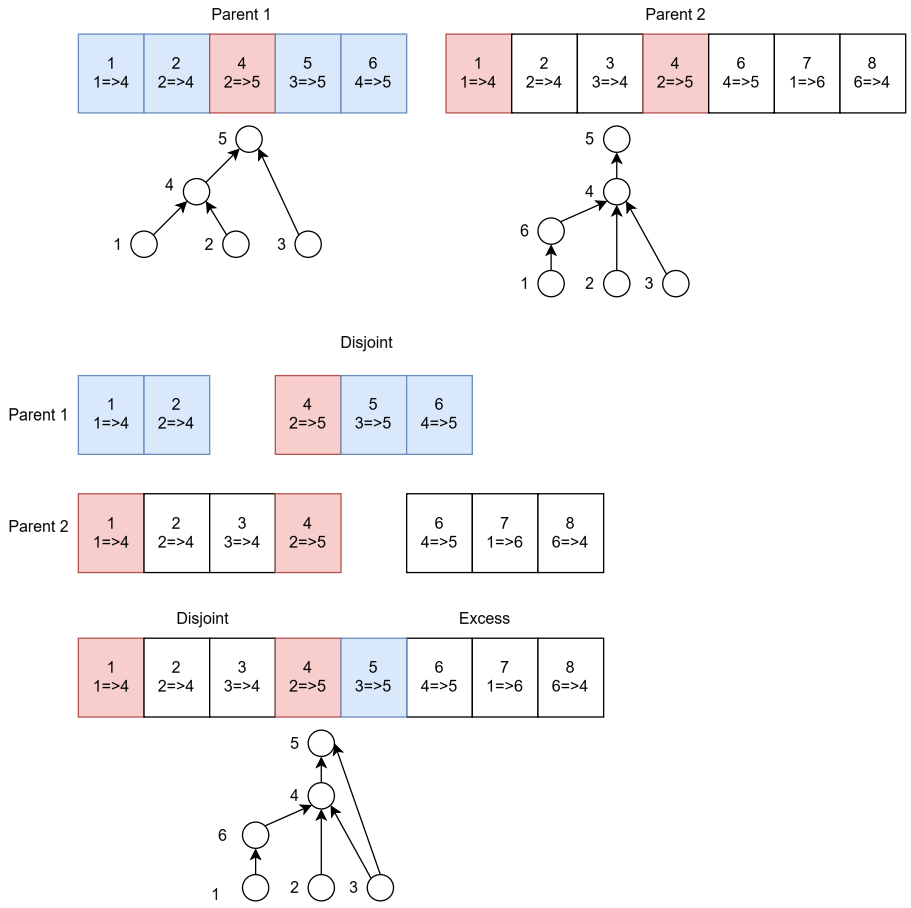


Figure 3.1: Figure was reproduced according to Stanley and Miikkulainen (2002). Matching up genomes for different network topologies using innovation numbers. Red squares represent disabled connections

.

3.2.4 Speciation and protection of structural innovations

To protect topological innovations, individuals of a population are divided into species based on a measure of the structural distance between genotypes.

$$\delta = \frac{c_1 \cdot E}{N} + \frac{c_2 \cdot D}{N} + c_3 \cdot \overline{W} \tag{3.8}$$

where E is the number of excess genes, D is the number of disjoint genes and W is the average deviation of the weights for matching genes. If the value of the distance between two individuals exceeds a set threshold parameter, the individuals belong to different species.

The fitness of each individual is subsequently adjusted through the mechanism of explicit division of fitness introduced by Goldberg and Richardson (1987): the value of

the fitness function of an individual is normalized by dividing by the number of its species. The calculations are given in the Formula (3.9)

$$f'_i = \frac{f_i}{\sum_{j=1}^{n} \mathrm{sh}(\delta(i,j))} \tag{3.9}$$

where i is the sh is the sharing function between individuals i and j. If the distance is lower than the threshold δ, the output of sharing function is set to 1 and otherwise 0. f_i is the fitness of the individual. n is the size of the population.

Thus, species with small numbers gain a relative advantage, which creates conditions under which structurally new individuals are able to compete mainly with their own species, rather than with the entire population, and consequently have a sufficient number of generations to optimize their own weight parameters before being subject to competition from more mature structures.

3.2.5 Minimal initialization of the structure

Unlike evolutionary approaches with random initialization of the initial population, in which individuals may initially have an excessively complex structure, in the NEAT algorithm, the population is initialized with topologically minimal networks. The complexity of the structure occurs gradually, with the accumulation of structural mutations that increase fitness. This approach is considered as one of the factors determining the effectiveness of the NEAT algorithm in the tasks of neuroevolution.

3.3 Changes to NEAT

While the original implementation of NEAT solved several of the problems for topological neuroevolution, it carries several structural imperfections that become increasingly problematic as evolution proceeds, particularly over longer runs or more complex search spaces. The following section examines a number of modifications to NEAT proposed by Lyubchev and Marchev (2025), each targeting one such limitation.

3.3.1 Accumulation of Weak, Near-Zero Connections

The original implementation of NEAT does not differentiate the structurally useful connections from connections with no contribution for the network performance. Because of that, the system often creates the links with the weights close to zero. Those connections

put the strain on the system, they are counted, stored, and evaluated each generation, and yet they pass no signal. Over the generations, population members accumulate those useless connections and become larger, more computationally expensive without any contribution to the system performance. According to Han et al. (2015) pruning connections identified as unimportant does not noticeably hurt network accuracy. This project implements following solution for the problem: after fitness evaluation the connection with weight smaller than a threshold τ is disabled, similar to pruning method used to compress large deep learning models by Han et al. (2015).

3.3.2 Loss of Elite Genomes

In the original paper NEAT maintains elitism only inside the species, which leads for the fittest individuals to be preserved only within their own species. As a consequence of this approach if a species is recognised as stagnant and removed from the population the best genotype is also discarded. This tendency has a potential to remove the best genome of the population or a genome containing crucial structural encoding for the future. Such behavior is potentially dangerous for the overall population fitness and can slow down the convergence of evolution Thierens and Goldberg (1994).

A population-wide elitism can be a desirable approach towards addressing this issue. This approach is preserving the best performing genotypes in a global list containing the best X genotypes. The members of the list are copied without any alteration from generation to generation and replaced with better performing individuals of the new generation. The list is not affected by survival of their respective species, therefore maintaining the highest fitness of the population.

Despite the advantages of preserving elitism it also introduces some risks. It increases the selection pressure, which carries a possibility of premature convergence. One of the ways to approach this issue is to keep the number of elite members relatively small.

3.3.3 Weight Incompatibility Between Matching Genes

Canonical NEAT solves the conflict between matching genes by randomly selecting the weight of one of the parents for the offspring. This approach is creating an issue of interrupting the behaviour of the genome. High performance of the population member can be a consequence of finely tuned behaviour. Random choice of the weight might discard the useful information from the parent, leading to slow convergence toward optimal solution.

Instead of copying a gene from a random parent, the authors propose merging the weight values of both parents into one. One of the ways to implement this suggestion is to use the Formula (3.10)

$$w_{\text{child}} = \alpha w_1 + (1 - \alpha) w_2 \tag{3.10}$$

where α is drawn from a uniform distribution $\mathcal{U}(0, 1)$ Lyubchev and Marchev (2025).

This combination approach allows offspring to inherit a weight that reflects both parents. Lyu et al. (2020) tested a similar blending mechanism and discovered that it lowers the probability of offspring having a broken configuration, while also preserving the consistency of improvement of the fitness across generations.

3.3.4 Inheritance Bias Toward the Fitter Parent

In the original implementation of NEAT all excess and disjoint genes are inherited from the fitter parent genetic crossover. Inheriting genes from superior genomes in theory leads to higher performance of the offspring, however practically it causes a problem of premature genetic conversion and reducing diversity Lyubchev and Marchev (2025). The approach for this problem introduced by Lyubchev and Marchev (2025) is to update crossover rules so that genes from less fit parents can be passed down to the next generation. Under the new rule, excess or disjoint genes coming from the less fit parent are included with probability P. As stated by authors, the most efficient value for P is 0.2 since with it the stronger genotype still has the most impact on the offspring generation and the weaker parent keeps the adequate diversity. According to Uriot and Izzo (2020), this type of strategies of inheriting genes from weaker parents help to preserve diversity by preventing one parent from taking over the crossover.

3.4 Proposed optimization method

3.4.1 Model architecture

The system was developed to simulate behavior of the communication heterogeneous network, given limited resources, uneven spread of users and constant movement of users. Improved NEAT algorithm is supposed to improve the network optimization by monitoring the current condition of the system and adjusting handover parameters accordingly.

The simulation performs on step by step model. Each step simulates following processes:

- Calculating the CIO control value for each station based on the NEAT output.
- Changing positions of the users.
- Calculating radio parameters for each user and station
- Choosing the serving station for each user and performing handover if one of the available stations is a better option
- Distributing physical resource block (PRB) to each connected user of the station and calculating user satisfaction, which is a ratio of received and requested PRB
- Marking network efficiency metrics

The model considers two types of base stations typical for the HetNet deployment:

- LTE stations provide wide coverage with relatively low bandwidth per user and low frequency
- 5G stations provide spot coverage of a small radius, but with significantly higher capacity due to the use of a high-frequency range

The following parameters are set in the model for each type of station:

Table 3.1: LTE and 5G Network Simulation Parameters

Parameter	**LTE**	**5G**	**Usage in simulation**
Coverage radius, m	500	200	Maximum distance at which a user is considered to be within the base station coverage area.
Transmit power, dBm	46	30	Used to calculate the received signal power (RSSI/RSRP)
Number of PRBs	100	2500	Determines the amount of available radio resources that can be allocated among users.
Carrier frequency, GHz	2.1	28	Used to calculate the path loss.
Bandwidth, MHz	20	500	Used to calculate the thermal noise and, consequently, the SINR.
Number of cells	7	21	Number of LTE and 5G base stations deployed in the simulation environment.

The positions of the stations is predetermined to ensure that the entire simulation map is covered and to preserve reproducibility of the results between runs.

The central LTE station is surrounded by 6 other LTE stations to provide uniform coverage of the map. The 5G stations are placed relative to one of the stations. For each LTE station a group of three 5G stations is created. This approach simulates a typical model in which operators install additional small 5G cells in the coverage area of an existing LTE cell to increase network capacity in a hotspots. Deveral additional 5G cells are placed along the edges of the simulation area to eliminate the blind spots of the network coverage on the border of the map. The stations placement is similar to the one used by Gures et al. (2023).

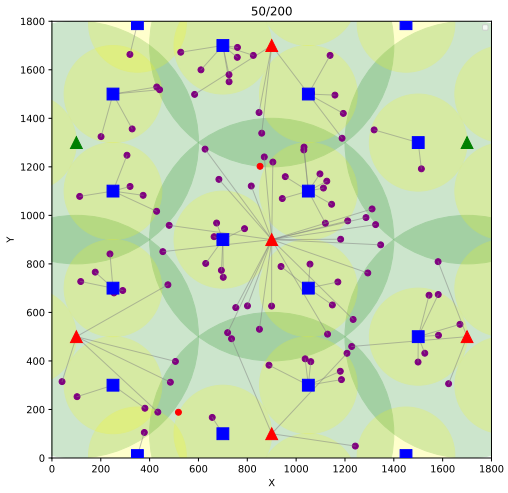


Figure 3.2: The construction of the map was inspired by Gures et al. (2023). Green triangles represent underloaded LTE stations, red triangles represent LTE station with load over 0.7, squares represent 5G stations, green transparent circles - radius of LTE stations, yellow transparent circles - radius of 5G stations, purple circles - connected users, red circles - disconnected users and black lines - connection between the user and the station.

.

To simulate the movement of the users a random target point mobility system is implemented. According to this system, each user receives at the initialization stage:

- the initial coordinates
- target coordinates

- the speed of movement
- the required traffic

Each step is assumed to simulate 0.5 seconds and at each step the following process is applied to each user:

- Calculation of the movement vector: $x_d = x_{target} - x$, $y_d = y_{target} - y$
- Calculation of Euclidean distance: $distance = \sqrt{x_d^2 + y_d^2}$
- Calculation of covered distance in one step: $speed_{ms} = \frac{speed_{km/h} \cdot 1000}{3600} \cdot 0.5$ (time for one step)
- If the distance to the target is lower than the distance covered in one step the user is considered reaching the target. After that new random target is generated, thus the movement continues without standing still.
- Each new position is saved in the list to be animated in the future.

3.4.2 Resource Allocation Model

Stations send information by information units, known as Physical Resource Block (PRB). The amount of data carried by one PRB depends spectral efficiency (se) value determined by SINR. In this project se calculation are simplified and determined by lookup table mapping SINR value between user and station to the se number Ghosh and Ratasuk (2011).

Each element of the table is representing a lower SINR bound, an upper SINR bound and the spectral efficiency value. If SINR of the user falls inside within the interval the corresponding se value is assigned. After that, for every user connected to a station, the model determines how many PRB would be needed to fully satisfy that user's requested traffic demand, given the spectral efficiency of the user.

The full algorithm can be desribed as:

- The user's current SINR is matched against the SINR-to-SE table to obtain the corresponding spectral efficiency value se.
- The throughput sent by one PRB is calculated as $\text{PRB}_{\text{thoughput}} = \text{PRB}_{\text{one}} \cdot \text{se}$, where PRB_{one} is the fixed bandwidth of a single resource block set to 180 kHz, according to 3GPP (2020)

- The number of PRB required to satisfy user's full demand is equal to round up value obtained by using the formula the formula $\text{user}_{\text{PRB}} = \frac{\text{traf}_{\text{req}} \cdot 1000000}{\text{PRB}_{\text{thoughput}}}$

The $user_{PRB}$ value is the ideal amount of PRB required to fully satisfy the user. Whether the station can allocate this ideal amount of PRB is determined by the number of connected users, their traffic demand and spectral efficiency.

Each base station has a finite total amount of PRBs detrmined by the type of station and is updated each step. The PRB allocation method distributes PRBs sequentially to each user currently connected to the station and monitors amount of PRB used and compares it to total number of available PRB.

There are three possible scenarios of resource allocation:

- The amount of PRB is enough to fully satisfy each connected user. If the users desired amount of PRB combined with amount of PRB already taken by other users does not exceed the station's available PRB, the user is granted full amount of requested PRB.

- Insufficient amount of resource blocks. If the station does not have enough PRB to satisfy user, the user receives only the amount of PRB available. The users satisfaction is the ration between the number of received prb and the requested number of PRB.

- Fully exhausted amount of available PRB. If station does not have any available PRB for a connected user, the algorithm assign equal amount of resources to each other $\text{PRB}_{\text{avg}} = \frac{\text{PRB}_{\text{total}}}{\text{N}_{\text{users}}}$, where N_{users} is the number of users connected to the station.

3.4.3 Serving-Cell Selection and Handover Algorithm

The decision of handover is made according to the RSRP value. If the users has better RSRP with the tower that is not the current one, they are connecting to the better station. However this system creates a problem of users not choosing 5G stations, as they have vastly more PRB than LTE towers, and consequently less RSRP. The Cell Individual Offset (CIO) value is an artificial value, created to solve this problem. The CIO value is added to the RSRP during the handover stage, increasing the possibility of users connecting to 5G towers. Rather than manually tuning the CIO values for each station this model delegates the computation of each station's CIO to a neural network whose parameters are optimized through NEAT, allowing the offset to be adapted dynamically.

At every step of the simulation, a set of available stations is determined for each user based on the distance. For each base station a combined value of CIO and RSRP is calculated and the user is connected to the station with the highest value.

Another important factor in handover decision is Time-to-Trigger (TTT), a value created to solve a problem of ping-pong effect. If the adjusted RSRP values of the neighboring cells are too close to each other, the user might be allocated to different towers each step of the simulation, interrupting connection.

To mitigate this issue, a TTT parameter is introduced. It determines the time during which the user cannot be allocated to the new station. This model implements TTT by adding fixed value of the simulation time to the counter of each user after allocation to the serving station. When the counter reaches certain threshold, user is allowed to be handover-ed to another station. Counter is set to 0 when new handover happens.

A handover is executed for a given user only when both of the following conditions hold simultaneously: the best station according to modified RSRP value is not current serving station and the users counter has exceeded the TTT threshold. If both conditions are satisfied then the user is removed from the serving station's connected users list and added to the list of new best station. The serving station id for the user is then updated and the handover data is stored in the handover history list for future signalizing.

The users are disconnected from their respective serving stations and their radio parameter are set to 0 if the distances between them and the serving stations are higher than their radius.

The total number of handovers is calculated and at the beginning of the next step sent to the NEAT as one of the input parameters, allowing the NN to be avoid too high or low number of handovers this step.

3.4.4 Neural network controlled CIO

In this project CIO value is controlled by neural network evolved by NEAT algorithm over hundreds of generations. At the beginning of the simulation step, the systems observed parameters are fed into the neural network as an input. For each station the NN receives a normalized input, describing different aspects of both local and the global system conditions:

- Station type describes whether the current serving station is 5G or LTE.
- Current serving station load

- Average system-wide station load
- Maximum load across all cells
- Serving station's connected user's average satisfaction
- System-wide average satisfaction
- Serving station average SINR
- System-wide average SINR
- The number of handover in the previous step

3.5 Other algorithms

In order to assess the performance of the proposed approach other algorithms has to be simulated. This section is dedicated to the explanation of the methodology behind approaches to compare.

3.5.1 Automated Weight Function algorithm

AWF system proposed by Gures et al. (2023) dynamically tunes key handover (HO) control parameters — the handover margin (HOM) and time-to-trigger(TTT) based on factors like the user's movement speed, RSRP value, and PRB availability.

The algorithm computes handover control parameters for each user separately. During the handover, the target station is selected based on the station's load and RSRP value. For maintaining accuracy of the metrics the load level of each station is recalculated each step the list of available stations is created. A station is added to the list if current user is within it's radios and the station's load is below 50% Gures et al. (2023). The target cell is then selected as the one with the highest RSRP among that candidate list.

The pseudo code describing the principle of work of the algorithm is depicted in Algorithm 1.

The algorithm is built around three bounded functions groups, each paired with its own auto-adjusted weighting:

- The first set of functions derives transitional values for the weights associated with user SINR, PRBs per unit, and unit velocity.

Algorithm 1 Pseudo code describing the AWF algorithm proposed by Gures et al. (2023)

```
Calculate SINR, number of PRBs per UE, and UE speed
if RSRP^target + HOM > RSRP^serving then
    if Trigger timer ≥ ΔTTT then
        HO decision ← True
        Send HO request
    else
        HO decision ← False
        Run Trigger timer
    end if
else
    HO decision ← False
    Reset Trigger timer
end if
```

- The second set takes those transitional values as input and computes a weight for each one.
- The third combines both the weights and the transitional values to produce a final value, which feeds into the HOM calculation that is used to fine-tune HCP settings for each user.

The first group:

$$f_y = \frac{SINR_t - SINR_s}{SINR_{max}}, \quad -1 \leq f_y \leq 1 \tag{3.11}$$

$$f_{PRB} = \frac{PRB_t}{PRB_{total,t}} - \frac{PRB_s}{PRB_{total,s}}, \quad -1 \leq f_{PRB} \leq 1 \tag{3.12}$$

$$f_v = 2\log_2\left(\frac{v + v_{max}}{v_{max}}\right) - 1, \quad -1 \leq f_v \leq 1 \tag{3.13}$$

The second group:

$$Total = (1 - f_y) + (1 - f_{PRB}) + (1 - f_v) \tag{3.14}$$

$$w_y = \frac{f_y}{Total} \tag{3.15}$$

$$w_{PRB} = \frac{f_{PRB}}{Total} \tag{3.16}$$

$$w_v = \frac{f_v}{Total} \tag{3.17}$$

The third group:

$$w_t = w_y + w_{PRB} + w_v \tag{3.18}$$

$$w'_y = \frac{w_y}{w_t} \tag{3.19}$$

$$w'_{PRB} = \frac{w_{PRB}}{w_t} \tag{3.20}$$

$$w'_v = \frac{w_v}{w_t} \tag{3.21}$$

$$f_{y,prb,v} = f_y w'_y + f_{PRB} w'_{PRB} + f_v w'_v \tag{3.22}$$

The $f_{y,prb,v}$ value is then used to calculate ΔHOM.

$$\Delta HOM = \begin{cases} \overline{HOM}\left(f(\gamma, PRB, v)\right), & \text{if } \gamma_{T,S} \leq \gamma_{th} \text{ or } \gamma_{T,S} \geq \gamma_{th}, \\ \overline{HOM}\left(1 + \omega_{PRB} f(PRB) + \omega_v f(v)\right), & \text{if } \gamma_T \leq \gamma_{th} \text{ and } \gamma_S \geq \gamma_{th}, \\ \overline{HOM}\left(-1 + \omega_{PRB} f(PRB) + \omega_v f(v)\right), & \text{if } \gamma_S \leq \gamma_{th} \text{ and } \gamma_T \geq \gamma_{th}. \end{cases} \tag{3.23}$$

Where $\overline{HOM}$ is calculated as shown in Formula (3.24) and $HOM_{\max}$, $HOM_{\min}$ are predefined parameters set to 20 and 0. y_t, y_s are SINR values of target and serving station.

$$\overline{HOM} = \frac{HOM_{\max} + HOM_{\min}}{2} \tag{3.24}$$

TTT values are continuously updated as follows in Formula (3.25)

$$\Delta T = \begin{cases} Z_1 & \text{if } T_{\min} < T < T_{\max}, \\ Z_2 & \text{if } T = T_{\min}, \\ Z_3 & \text{if } T = T_{\max}, \end{cases} \tag{3.25}$$

where Z_1, Z_2 and Z_3 are represented by Formulas (3.26) (3.27) (3.28) and T is current Time-To-Trigger:

$$Z_1 = \begin{cases} T - \rho & \text{if } f_{WF} \leq f_{WF} + \mathcal{Q}, \\ T + \rho & \text{if } f_{WF} \geq f_{WF} + \mathcal{Q}, \end{cases} \tag{3.26}$$

$$Z_2 = \begin{cases} T & \text{if } f_{WF} \leq f_{WF} + \mathcal{Q}, \\ T + \rho & \text{if } f_{WF} \geq f_{WF} + \mathcal{Q}, \end{cases} \tag{3.27}$$

$$Z_3 = \begin{cases} T - \rho & \text{if } f_{WF} \leq f_{WF} + \mathcal{Q}, \\ T & \text{if } f_{WF} \geq f_{WF} + \mathcal{Q}. \end{cases} \tag{3.28}$$

For simplicity, the values of ρ and $\mathcal{Q}$ are set as 0.04 and 0.1 s. Additionally, T_{min} and T_{max} are determined as 0.0 and 5.12 s.

The evaluation of target and serving cell are depicted as $RSRP_{pilot}$. Values are calculated separately as $RSRP_{pilot,s}$ and $RSRP_{pilot,t}$. Each value is obtained using formula:

$$RSRP_{pilot} = RSRP \cdot \frac{PRB_{total}}{\max\left(N_{users},\ 1\right)} \tag{3.29}$$

If the sum of $RSRP_{pilot,t}$ and ΔHOM is higher that $RSRP_{pilot,s}$, then conditions for handover are fulfilled.

3.5.2 RL-based model

Other algorithm to compare with is Rl-based model proposed by Attiah et al. (2020) that optimizes neighboring cell parameters for improved load balancing within a geographical cluster of stations. In this method handover from station i to station j is triggered if the following conditions is fulfilled (3.30)

$$M_j + \theta_{j \to i} > \text{Hys} + M_i + \theta_{i \to j} \tag{3.30}$$

where M_i, M_j are RSRP values, θ is the CIO value and Hys is the hysteresis margin. For simplicity each station has a single CIO value applied uniformly to all of its neighbors.

The algorithm proposed in the paper Attiah et al. (2020) was designed for localized geographical clusters of 3 stations. Due to the high computational cost mentioned in literature review section of this paper the original method can not be implemented directly to the simulation. To solve this issue the stations has been divided into blocks consisting of LTE station and nearest 5G stations. A separate RL agent is trained for each block, reducing the dimensionality of the action space compared to a fully centralized model.

For a block of n_b stations, the state $S(t)$ is constructed as the combination of three components, as described by Attiah et al. (2020):

$$S(t) = \left[U(t)^T,\ R(t)^T,\ \text{vec}(M(t))^T \right]^T \tag{3.31}$$

Where:

- $U(t) \in [0,1]^{n_b}$ is the load of each station

- $R(t) \in R_+^{n_b}$ is the normalized average throughput of users at the station
- $M(t) \in [0,1]^{n_b \times \mu}$ is an indicator of what fraction of a station's connected users fall into each spectral efficiency level.

Input dimensionality: $2n_b + n_b\mu$.

The agent's action is a CIO vector for the block stations:

$$A(t) = (\theta_1(t), \ldots, \theta_{n_b}(t)), \qquad \theta_n(t) \in \{-100, -50, 0, 50, 100\} \tag{3.32}$$

The total number of action combinations per block:

$$L^{n_b} \tag{3.33}$$

where L is equal to 5.

Each block has its own fully connected network with 3 hidden layers, each of size max $(2n_b + n_b\mu, L^{n_b})$, ReLU activation function and Dropout $q = 0.5$; the output layer has size L^{n_b} with softmax:

$$\sigma(z_i) = \frac{exp(z_i)}{\sum_{j=1}^{L^{n_b}} exp(z_j)} \tag{3.34}$$

where z_i is the an input to the output layer of the neural network.

The output of the network is a probability distributor for actions from which a random action with probability of $\epsilon(t)$ and the action that maximizes the predicted Q-function is exploited with probability 1-$\epsilon(t)$

$$A(t) = \begin{cases} \arg\max_A Q_t(S(t), A) & \text{w.p. } 1 - \epsilon(t) \\ A_{\text{random}} & \text{w.p. } \epsilon(t) \end{cases} \tag{3.35}$$

where A_{random} is drawn uniformly from the action space.

$\epsilon(t)$ is exploration probability:

$$\epsilon(t) = \epsilon_d^{\ell(t)} \tag{3.36}$$

where $\epsilon_d = 0.995$ is decay factor of the exploration probability and $\ell(t)$ is the index of the time step corresponding to t.

$$R(t) = \sum_{n=1}^{N} \sum_{k_n=1}^{K_n} R_{k_n}(t) \tag{3.37}$$

where $R_{k_n}(t)$ is the measured throughput of the user in the n'th station at time t. The agent agent implements a stochastic policy π_t which is probability of performing the

action A(t)=a given it was in state S(t)=s. The agent's goal is to maximize the long-term average reward:

$$\max_{\pi} \lim_{L\to\infty} E\left[\frac{1}{L}\sum_{t=0}^{L} R(t)\right] \tag{3.38}$$

The action A(t) is used and reward R(t+1), state S(t+1) are observed. The Q-function is updated according to the Formula (3.39)

$$Q_{t+1}(S(t), A(t)) = R(t+1) + \lambda \max_{a} Q_t(S(t+1), a) \tag{3.39}$$

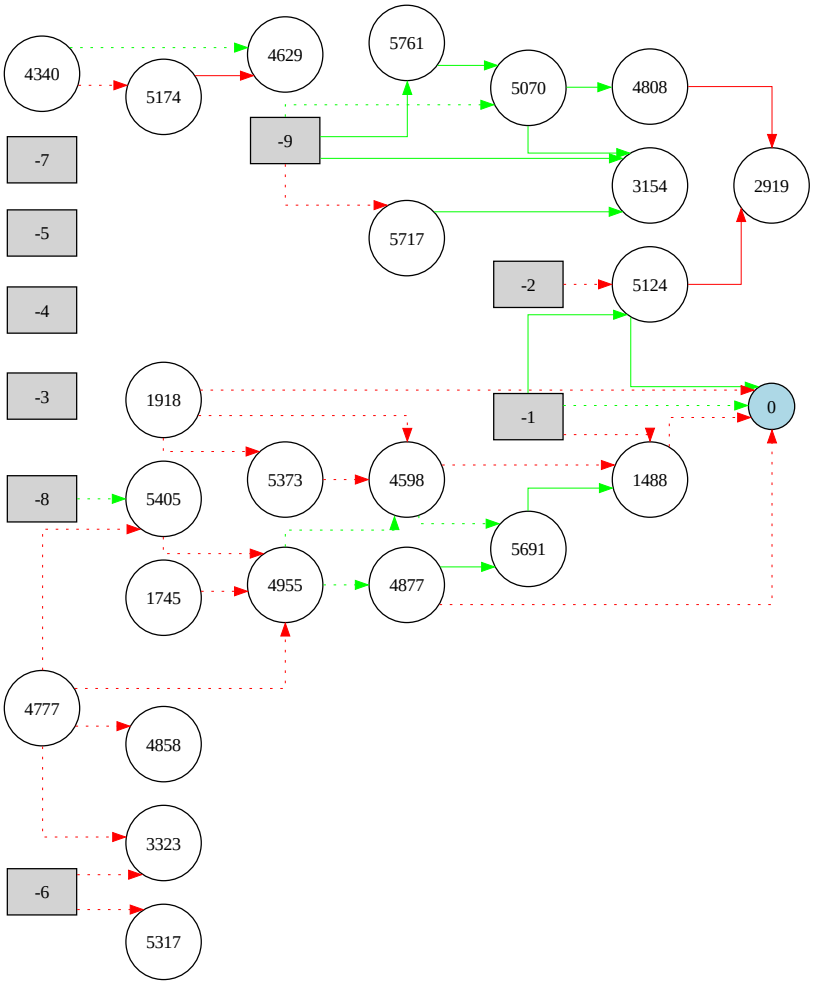


Figure 3.3: Graphical representation of the network. Gray squares represent input neurons, empty circles are hidden neurons and blue circle is output. Green lines have positive weights and red lines - negative. Dotted lines are disabled connections and straight lines are active. As presented in the figure, only one connection to the output is active and it is coming from the node that itself is receiving signals only from input -1.

Chapter 4

Results

4.1 Performance of NEAT-based load balancing algorithm

4.1.1 Simulation setup

The evaluation of the NEAT-based load balancing algorithm was carried out during the evolution stage using the simulation parameters listed in the Table 4.1.

Table 4.1: Simulation parameters

Parameter	**Value**
Number of users	100
Simulation area	1800×1800 m
Initial user position	Random, 0–1800 m
User target position	Random, 0–1800 m
User speed	1–160 km/h
Traffic demand	1–8 Mbps
Simulation time step	0.5 s
Number of simulation steps	200
Number of generations	500

For fair comparison, all population members of all runs were tested under the same conditions: 100 users were located in a vertical line at the right end of the map. During the simulation they moved at the same speed of 130 km/h in a horizontal line towards the left end of the map. When the users reached the left end of the map, they started moving back to the right end. All users had the same traffic demand of 5 Mbps. All networks evolved over 500 generations. The evolution process for the networks lasted 10-12 hours. The Tables 4.3 and 4.5 represent the best results of the runs.

The fitness is calculated according to the Formula (4.1). All values are normalized as fractions of maximum possible value.

$$\begin{aligned} \text{fitness} = 2 \cdot \text{satisfaction}_{\text{avg}} + \text{load}_{\text{5G,avg}} - \text{load}_{\text{LTE,avg}} \\ - 0.2 \cdot \text{handovers}_{\text{total}} - \text{load}_{\text{max}} - 3 \cdot \text{disconnected}_{\text{total}} \end{aligned} \tag{4.1}$$

4.1.2 NEAT N1

The key parameters of the NEAT configuration N1 are listed in Table 4.2.

Table 4.2: NEAT configuration N1 parameters

Parameter	**Value**
Fitness criterion	max
Fitness threshold	3.9
Population size	50
Activation default	tanh
Activation options	tanh
Aggregation default	sum
Initial connection	full direct
Feed forward	True
Number of inputs	9
Number of outputs	1
Number of hidden nodes	0
Connection addition probability	0.45
Connection deletion probability	0.20
Node addition probability	0.45
Node deletion probability	0.20
Weight mutation rate	0.80
Weight mutation power	0.50
Weight replacement rate	0.10
Weight initial standard deviation	0.50
Bias mutation rate	0.70
Bias mutation power	0.50
Bias replacement rate	0.10
Compatibility disjoint coefficient	1.0

Parameter	Value
Compatibility weight coefficient	0.5
Compatibility threshold	3.0
Maximum stagnation	20
Species elitism	2
Elitism	2
Survival threshold	0.2
Minimum species size	2

Results of NEAT evolution with configuration N1 are presented in the Table 4.3.

Table 4.3: NEAT configuration N1 results. Traffic is not included in the fitness function because it is a reflection of satisfaction.

Fitness	Traffic	Satisfaction	Disconnected users	Time (s)
-0.337644	2.457912	0.481892	150	2.4480
-0.337644	2.457912	0.481892	150	2.4456
-0.333864	2.431867	0.476806	134	2.4444
-0.337644	2.457912	0.481892	150	2.4642
-0.334629	2.442943	0.478974	138	2.4480
-0.331331	2.474992	0.485427	164	2.4525
-0.334217	2.445318	0.479421	139	2.4517
-0.335482	2.451276	0.480316	143	2.4469
-0.333927	2.436842	0.477583	136	2.4552
-0.335108	2.449637	0.479874	141	2.4438

4.1.3 NEAT N2

To assess the algorithm's sensitivity to evolution parameters, a second configuration was tested. Compared to N1, N2 uses lower structural-mutation probabilities and introduces bounds on the range of weights and biases. Only the parameters that differ from N1 are listed in Table 4.2. All remaining parameters are identical to those in Table 4.4.

Table 4.4: Selected NEAT mutation parameters

Parameter	**Value**
Connection addition probability	0.25
Node addition probability	0.25
Weight mutation rate	0.60
Weight mutation power	0.30
Weight maximum value	10
Weight minimum value	-10
Bias mutation rate	0.60
Bias mutation power	0.30
Bias initial standard deviation	1.0
Bias maximum value	10
Bias minimum value	-10

Results of NEAT evolution with configuration N2

Table 4.5: NEAT configuration N2 results

Fitness	**Traffic**	**Satisfaction**	**Disconnected users**	**Time (s)**	**Ping-Pong**
-0.332420	2.435775	0.477520	136	2.4604	0
-0.332471	2.435720	0.477506	136	2.5431	0
-0.335795	2.464387	0.482504	150	2.5075	0
-0.337644	2.457912	0.481892	150	2.5186	0
-0.337446	2.456827	0.481674	150	2.4616	0
-0.336955	2.459544	0.482213	150	2.5007	0
-0.333185	2.441236	0.478421	138	2.4873	0
-0.334012	2.449873	0.480156	142	2.4958	0
-0.335326	2.453614	0.481037	146	2.4736	0
-0.336218	2.458927	0.481854	148	2.5291	0

A comparison of the results obtained with configurations N1 and N2 showed that the differences between them were insignificant. Accordingly, configuration N1 was selected for the subsequent experiments.

4.1.4 Recurrent neural network (RNN)

Recurrent neural networks use recurrent connections, where the output of a neuron at one time step is fed back as input to the network at the next time step. This allows RNNs to capture temporal dependencies and patterns within sequences. Recurrent networks evolved over 1000 generations — twice as many as the non-recurrent networks.

The results of using recurrent connections with NEAT are shown in Table 4.6.

Table 4.6: Recurrent neural networks

Fitness	Traffic	Satisfaction	Disconnected users	Time (s)	Ping-Pong
-0.333546	2.430570	0.476932	134	2.4392	0
-0.333481	2.446570	0.479710	140	2.4535	0
-0.334386	2.440766	0.478546	138	2.5355	0
-0.337643	2.457912	0.481891	150	2.5462	0
-0.335617	2.458637	0.481878	144	2.4413	0
-0.333486	2.449178	0.480222	140	2.4963	0
-0.334012	2.443925	0.479184	139	2.4876	0
-0.334721	2.451836	0.480631	142	2.4718	0
-0.335184	2.454672	0.481143	145	2.5184	0
-0.336027	2.456183	0.481507	147	2.5291	0

4.1.5 Random simulation testing

To verify the robustness and generalization ability of the resulting models, all networks were additionally tested under random simulation conditions similar to those used during the evolution stage. Each network was tested 20 times to assess the accuracy of the results.

The results of random testing of the non-recurrent models are shown in Table 4.7.

Table 4.7: Results of random testing of non-recurrent models

Fitness	Traffic	Satisfaction	Disconnected users	Time (s)	Ping-Pong
-0.427518	2.982175	0.694175	70	2.3413	0
-0.456301	2.861139	0.676345	62	2.3841	0
-0.393877	3.137984	0.697599	87	2.3635	0
-0.437500	3.340971	0.695075	132	2.3562	0
-0.399538	3.295940	0.714261	118	2.3503	0

Fitness	Traffic	Satisfaction	Disconnected users	Time (s)	Ping-Pong
-0.438864	3.196393	0.687307	106	2.3525	0
-0.421763	3.084517	0.701428	91	2.3657	0
-0.445218	3.218764	0.681936	123	2.3731	0
-0.409684	3.176352	0.706815	98	2.3489	0
-0.432951	3.251087	0.692574	109	2.3598	0

Although satisfaction is higher and the number of disconnected users is lower when compared to static test, the fitness is consistently lower. The suggested reason is the user behavior in the static experiment. They often leave the radios of the stations increasing the number of handovers. In contrast, during random simulation a considerable amount of users tend to move inside one stations area for a long time, reducing the amount of handovers and increasing the fitness. The example is given in the Figure 4.1 and 4.2

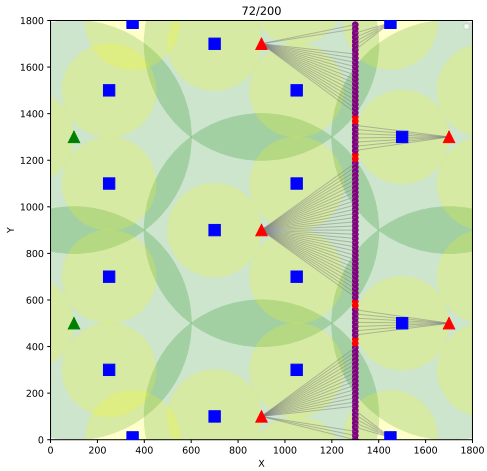


Figure 4.1: Behavior of the users during static simulation

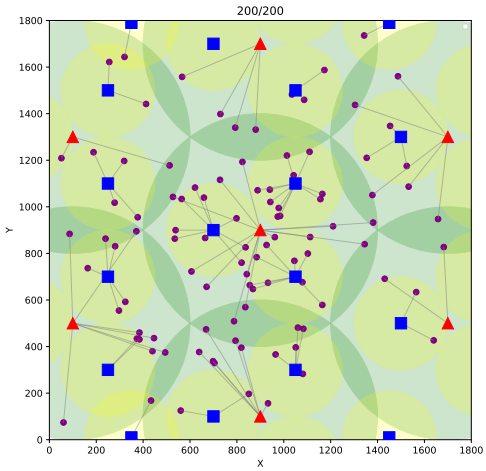


Figure 4.2: Behavior of the users during random simulation

4.1.6 Fitness and speciation during static testing

The results in Figure 4.3 and 4.4 suggest that in the chosen conditions the models quickly converge to the same solution and after 250 generation genotypes stop dividing into a new species. This suggestion aligns with the structure of the graphs. Examples are given in Appendix A.2. In the Figure 3.3 input -1 is mapped to the output through only one neuron and no other connections to the output are active. Other population members evolved in a similar pattern or the pattern in which input -1 is directly mapped to the

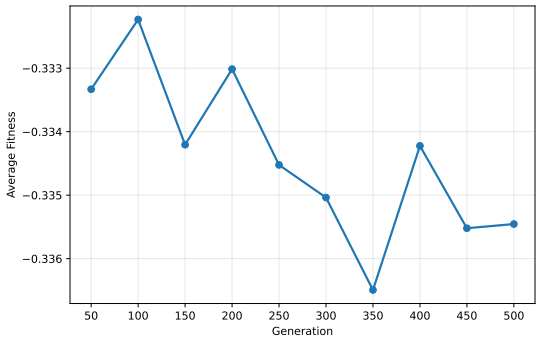


Figure 4.3: Average fitness over generations

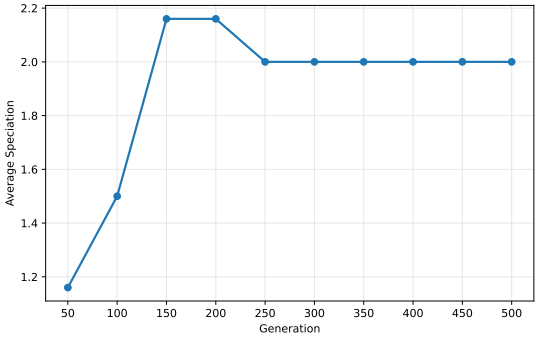


Figure 4.4: Average speciation over generations

output.

4.1.7 Analysis of the performance of the NEAT-based load balancing algorithm

According to the results of the simulations, fitness values of various network align in the same bounds within -0.331 and -0.338. Results of the testing various configurations suggest that neural networks evolved via NEAT converge broadly to similar solution. Configuration N1 exlored search space more aggressively but did not achieve meaningfully better outcome than more conservative N2. Models differ only marginally suggesting that for the current problem the evolution is not highly sensitive to the tuning.

Recurrent connections did not produce noticeably different results from feedforward even after 1000 generations. All performance metrics fall within the same boundaries as the non-recurrent results. This results implies that advantages of keeping information of the previous step were not useful for particular problem of load optimization. The suggested reason is that input layer of the NN already provide sufficient amount of information.

Ping-pong metric across all networks is 0, meaning that the method achieved the goal of avoiding excess handovers.

All models evolved in a pattern to ignore any inputs other than station type. The suggested reason is combination of fitness formula directly enforcing higher load on the 5G stations and lower load on LTE stations and incremental nature of NEAT in which all genotypes start with all inputs directly connected to the output. The latter means that all genotypes most likely will eventually explore this pattern.

All the models use a simplified heuristic instead of a strategy that takes into account the current load on a specific cell, the number of users, the frequency band, etc. Such a

simplified rule may not generalize to cases where a specific 5G cell is already overloaded, while a neighboring LTE cell is free. The insufficient use of the other eight inputs may be a separate limitation of the found solution.

4.2 Comparison with reinforcement learning and automatic weight function methods

To compare the proposed algorithm against alternative load balancing approaches, two additional algorithms from the literature were re-implemented and evaluated under the same simulation conditions: a reinforcement learning algorithm that optimizes the CIO of each cell using Q-learning Attiah et al. (2020) and an Automatic Weight Function (AWF) algorithm that dynamically tunes the handover margin and TTT based on SINR, PRB availability and user speed Gures et al. (2023). Both algorithms were evaluated 20 times under random test conditions and once under the static test scenario, using the same static scenario as the one used for the NEAT models. For reference, the corresponding NEAT figures below are the averages of the 10 models reported in Table 4.3 and Table 4.7. RL and AWF algorithmas were compared to the highest performing member of the populations evolved by NEAT.

Table 4.8: Comparing proposed algorithm with RL-based and AWF-based in random simulation conditions

Metric	NEAT (avg.)	Reinforcement Learning	Automatic Weight Function
Traffic	3.154532	0.786	2.200
Satisfaction	0.694752	0.132	0.412
Disconnected users	99.6	98.0	89.65
Time	2.35954	12.24	5.76
Ping-Pong	0	0	324.5

Table 4.9: Comparing proposed algorithm with RL-based and AWF-based in static simulation conditions

Metric	NEAT (avg.)	Reinforcement Learning	Automatic Weight Function
Traffic	2.451	0.476	2.080
Satisfaction	0.480	0.086	0.295

Metric	NEAT (avg.)	Reinforcement Learning	Automatic Weight Function
Disconnected users	144.5	143.2	150
Time (s)	2.45	11.868	6.54
Ping-Pong	0	0	449

4.2.1 Analysis of the performance of proposed algorithm compared to Rl-based and AWF-based methods

According to the obtained results, NEAT achieves the best fitness and is consistent over both simulation types, producing almost equally optimal behavior. In contrast, RL and AWF models perform differently on random and static simulations.

RL algorithm performs worse compared to NEAT in terms of traffic and satisfaction and the average time for simulation is far greater, leading to much slower execution in real time scenario. Although this method performs best in terms of lowering the number of disconnected users, this result is achieved by lowering the traffic for the users.

AWF method is closer to NEAT in terms of traffic, however it is the only algorithm that has any Ping-Pong value and the value is very high. In the original paper similar results were not presented. A high Ping-Pong value implies a large potion of users being handed back and forth between stations every few steps. It is possible that this behavior is an indication of decision process not being simulated correctly.

4.2.2 Limitations

Both RL and AWF methods were implemented based on the original publications and are not the exact copies of the codes developed by authors. A part of relatively low performance compared to NEAT-based approach is likely because of the technical implementation. The time constrains of the project does limits the amount of time and effort available for tuning and developing algorithms, as most of it is dedicated towards proposed algorithm.

Chapter 5

Conclusion

The study was aimed at verifying whether the NEAT-based load balancing algorithm is capable of effectively optimizing the handover process in heterogeneous networks. The following conclusion was written according to the results.

This paper has two research questions:

1. Can a NEAT-based controller effectively optimize handover parameters for load balancing in heterogeneous cellular networks?

2. Does NEAT improve load balancing performance compared to conventional fixed-topology neural networks (and other established methods)?

Answering the first question, the results confirm that a NEAT-based controller can effectively optimize the handover process. Configuring the parameters of the neuro evolution did not affect the performance. This indicates that, for this problem the evolution process is weakly sensitive to the tuning of NEAT hyperparameters. Similarly, the use of recurrent connections and doubling the number of evolution generations did not lead to a significant improvement in the metrics. This indicates that information about previous time steps does not add additional value to solving the load balancing problem. Both tuning and recurrent connection were inefficient probably because the networks evolved to use station type as the main factor in decision making.

Answering the second question, a comparison with the RL approach of Attiah et al. (2020) and the AWF method of Gures et al. (2023) under identical conditions suggests advantages for the NEAT approach. In terms of traffic and user satisfaction, NEAT outperformed both alternative methods under both random and static conditions — for example, achieving a satisfaction score of 0.695 versus 0.132 for RL and 0.412 for AWF in random tests. NEAT was also significantly faster in execution time, which is important

for real-time deployment. The AWF method showed extremely high Ping-Pong values, indicating instability in its decision-making process, possibly due to inaccuracies in the implementation relative to the original publication. The RL method achieved the lowest number of disconnected users, but at the cost of a significant reduction in traffic compared to NEAT-based approach. Both the RL and NEAT-based approaches successfully avoided the Ping-Pong effect, which was one of the goals of the algorithms.

Analysis of the structure of the evolved networks showed that the models are relying primarily on the station type. As a result, the solution found is a simplified heuristic that may not generalize to situations where a specific 5G cell is already overloaded, while the neighboring LTE cell is free.

This paper is written under several limitations, including time constraint and simplified simulation. Due to the limited time, RL and AWF based models were implemented on a surface level, possibly causing performance struggles. This implies that one of the possible reasons for NEAT-based approach's comparatively high performance is incorrectly implemented analogs. Another consequence of limited time is limited amount of simulation conditions, maps, user models, etc. It is supposed that the limited environment was the main reason for models to rely on the station type.

The simulation was simplified by using simple thermal noise model assuming constant temperature, no other sources of noise and no obstacles or reflections.

The results provide evidence that evolutionarily developed neural networks based methods could be a viable alternative to RL and AWF approaches to load balancing in heterogeneous networks. However reliance of the models of a single parameters remains concerning. Future work in this direction should evolve proposed algorithm in more realistic conditions, in which such a strategy will be infective, with a model that accounts for obstacles, shadowing, and various environmental noise. The algorithm's performance should be compared against a broader set of load balancing methods and explore the effectiveness of the modifications to the NEAT proposed by Lyubchev and Marchev (2025).

Bibliography

3GPP (2020). LTE; Evolved Universal Terrestrial Radio Access (E-UTRA); Physical channels and modulation. Technical Specification TS 36.211 V16.2.0, 3rd Generation Partnership Project. 16

Agarwal, A., Khan, S., Mohapatra, S. S., and Sahoo, S. K. (2026). Performance evaluation of sinr in 5g urban macro-cells with variable parameters under different path loss models. *Journal on Wireless Communications and Networking*, 2026(1). 6

Ansari, S., Boutaleb, T., Sinanovic, S., Gamio, C., and Krikidis, I. (2017). Mhav: Multitier heterogeneous adaptive vehicular network with lte and dsrc. *ICT Express*, 3(4):199–203. 1

Attiah, K., Banawan, K., Gaber, A., Elezabi, A., Seddik, K., Gadallah, Y., and Abdullah, K. (2020). Load balancing in cellular networks: A reinforcement learning approach. In *2020 IEEE 17th Annual Consumer Communications & Networking Conference (CCNC)*, pages 1–6. iii, 1, 2, 3, 22, 33, 35

European Telecommunications Standards Institute (2025). Lte; evolved universal terrestrial radio access (e-utra); physical layer; measurements. Technical Report ETSI TS 136 214 V19.0.0; 3GPP TS 36.214 version 19.0.0 Release 19, European Telecommunications Standards Institute. 5, 6

Farré, V., Vega-Sánchez, J., Garzón, V., Garzón, N. O., Mora, H. C., and Benitez Olivo, E. E. (2025). Design and performance analysis of sub-thz/thz mini-cluster architectures for dense urban 5g/6g networks. *Sensors*, 25(21):6717. 6

Franco, C. A. S. and De Marca, J. R. B. (2015). Load balancing in self-organized heterogeneous lte networks: A statistical learning approach. In *2015 7th IEEE Latin-American Conference on Communications (LATINCOM)*, pages 1–5. 2

Ghosh, A. and Ratasuk, R. (2011). *Essentials of LTE and LTE-A*. Cambridge University Press. 16

Goldberg, D. E. and Richardson, J. (1987). Genetic algorithms with sharing for multimodal function optimization. In Grefenstette, J. J., editor, *Proceedings of the Second International Conference on Genetic Algorithms*, pages 148–154, San Francisco, California. Morgan Kaufmann. 10

Gomez, F. and Miikkulainen, R. (1997). Incremental evolution of complex general behavior. *Adaptive Behavior*, page 5:317–342. 4

Gruau, F., Whitley, D., and Pyeatt, L. (1996). A comparison between cellular encoding and direct encoding for genetic neural networks. In Koza, J. R. et al., editors, *Genetic Programming 1996: Proceedings of the First Annual Conference*, pages 81–89, Cambridge, Massachusetts. MIT Press. 4

Gures, E., Shayea, I., Saad, S. A., Ergen, M., El-Saleh, A. A., Ahmed, N. M. S., and Alnakhli, M. (2023). Load balancing in 5g heterogeneous networks based on automatic weight function. *ICT Express*, 9(6):1019–1025. iii, viii, 3, 15, 19, 20, 33, 35

Han, S., Pool, J., Tran, J., and Dally, W. J. (2015). Learning both weights and connections for efficient neural networks. In *Advances in Neural Information Processing Systems*, volume 28, pages 1135–1143. 12

Holma H., . T. A. (2012). *LTE Advanced: 3GPP Solution for IMT-Advanced*. John Wiley & Sons. 1

Lobinger, A., Stefanski, S., Jansen, T., and Balan, I. (2010). Load balancing in downlink lte self-optimizing networks. In *2010 IEEE 71st Vehicular Technology Conference*, pages 1–5. 2

Lyu, Z., ElSaid, A., Karns, J., Mkaouer, M., and Desell, T. (2020). An experimental study of weight initialization and weight inheritance effects on neuroevolution. *arXiv*. 13

Lyubchev, D. and Marchev, A. (2025). Revisiting and improving the neat algorithm. In *2025 IEEE International Conference on Systems, Man, and Cybernetics (SMC)*, pages 3698–3705. 4, 11, 13, 36

Stanley, K. O. and Miikkulainen, R. (2002). Evolving neural networks through augmenting topologies. *Evolutionary Computation*, 10(2):99–127. viii, 2, 4, 7, 10

Thierens, D. and Goldberg, D. E. (1994). Elitist recombination: An integrated selection-recombination genetic algorithm. In *Proceedings of the IEEE World Congress on Computational Intelligence*, pages 508–512. 12

Uriot, T. and Izzo, D. (2020). Safe crossover of neural networks through neuron alignment. In *Proceedings of the Genetic and Evolutionary Computation Conference*, pages 957–965. 13

Appendix A

Appendix

A.1 Instruction for running the code

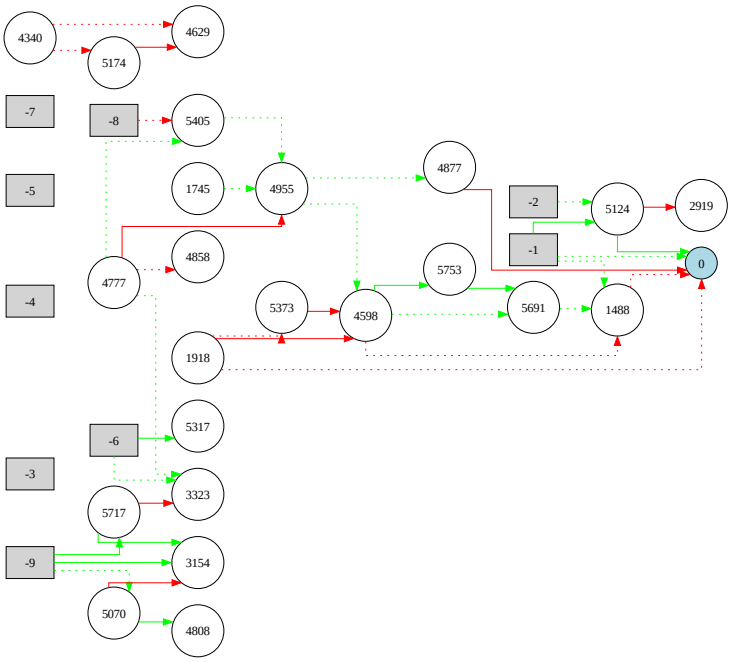
Necessary files and the updated neat-python library are included in the uploaded .zip file. The following libraries must be downloaded:

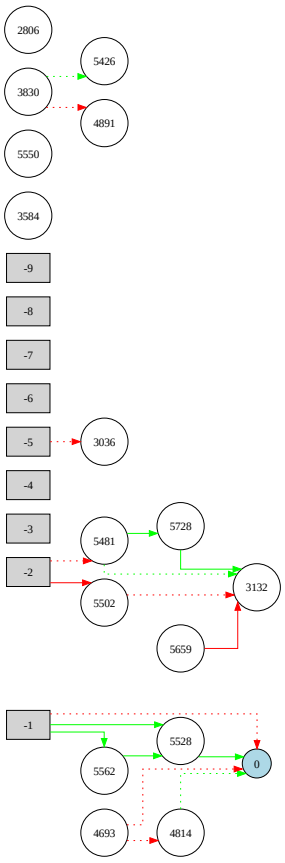
- numpy
- networkx
- matplotlib
- graphviz
- torch

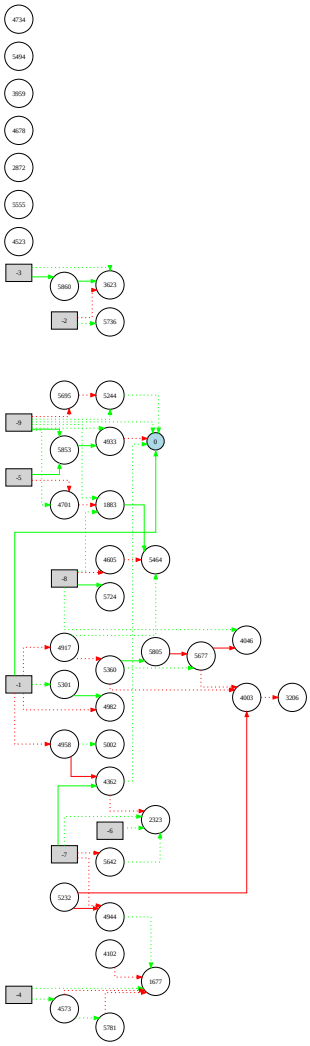
Updated neat must be placed in the same folder as the other libraries.

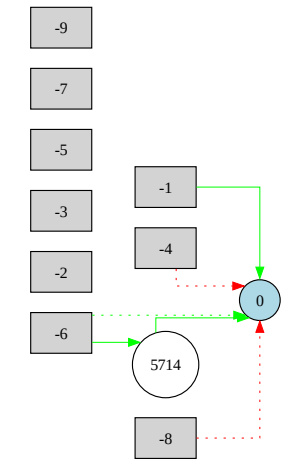
In order to run the code for the proposed algorithm, the ‘balance_neat.py‘ file has to be executed. By default, the code runs the ‘test_1‘ method, which tests the selected population. The ‘test_1‘ method calls the ‘run_simulation‘ method in the ‘course.py‘ file. This file tests both NEAT and RL-based approaches. To choose between them, the corresponding line of code in the ‘run_simulation‘ method has to be commented out. By default, the NEAT method is selected. The AWF method is tested separately using the ‘AWF.py‘ file.

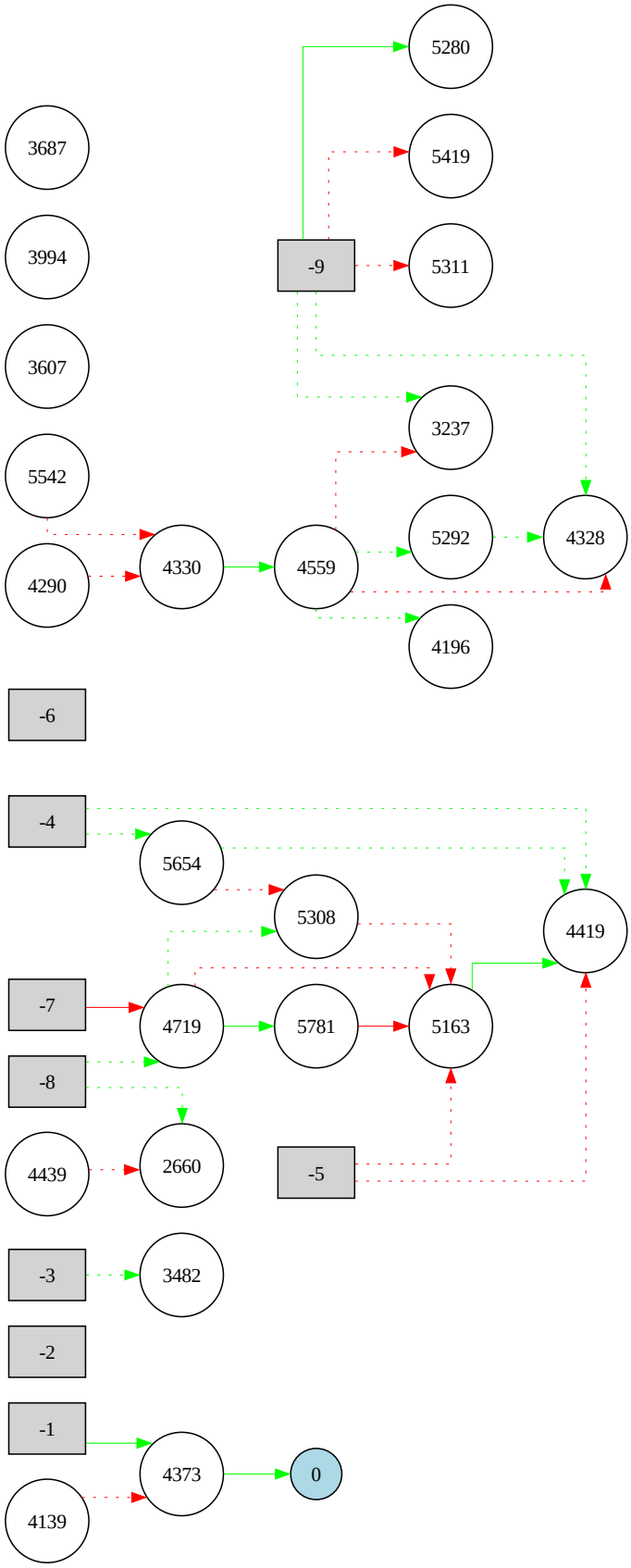
A.2 Topology examples

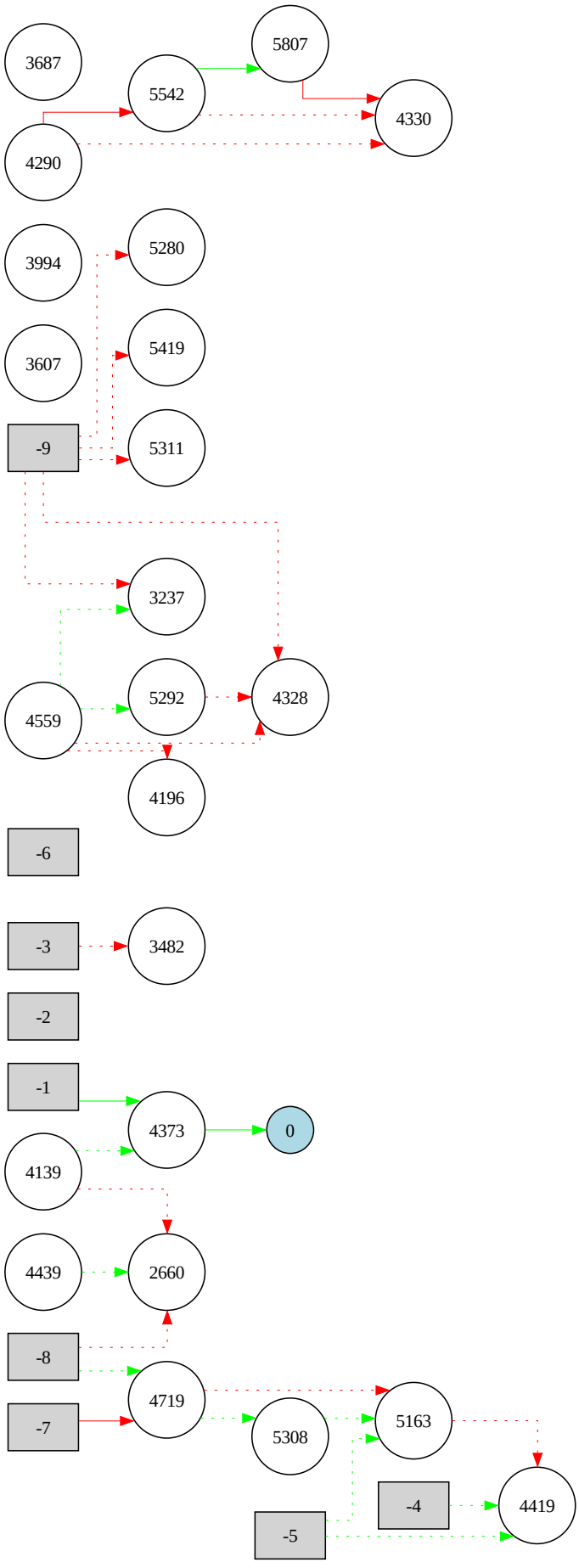

4340
4629
5174
-7
-8
5405
-5
1745
4955
4877
-2
5124
2919
-1
0
4858
4777
5753
-4
5373
4598
5691
1488
1918
5317
-6
-3
3323
5717
-9
3154
5070
4808


2806
5426
3830
4891
5550
3584
-9
-8
-7
-6
-5
3036
-4
-3
5481
5728
-2
3132
5502
5659
-1
5528
0
5562
4693
4814


4734
5494
3959
4678
2872
5555
4523
-3
5860
3623
-2
5736
5695
5244
-9
5853
4933
0
-5
4701
1883
4605
5464
-8
5724
4917
4946
5805
5677
5360
-1
5301
4003
5206
4982
4958
5002
4362
2323
-6
-7
5642
5232
4944
4102
1677
-4
4573
5781


-9
-7
-5
-3
-2
-6
-1
-4
0
5714
-8


5280
3687
5419
3994
-9
5311
3607
3237
5542
5292
4328
4330
4559
4290
4196
-6
-4
5654
5308
4419
-7
4719
5781
5163
-8
4439
2660
-5
-3
3482
-2
-1
4373
0
4139


3687
5807
5542
4330
4290
3994
5280
3607
5419
-9
5311
3237
4559
5292
4328
4196
-6
-3
3482
-2
-1
4373
0
4139
4439
2660
-8
4719
-7
5308
5163
-4
4419
-5